KUROSHIRO

Project Report & User Manual

1. Game Rules & Mechanics

This game is a unique variant of chess where standard pieces are replaced by custom units featuring special abilities, health pools, and distinct combat mechanics.

1.1 Win Condition

Unlike standard chess which ends in checkmate, this game relies on total annihilation. You win the game when the opponent loses all of their pieces.

1.2 Custom Pieces

- **Warrior:** The frontline unit. It advances like a standard Pawn and serves as the team's tank, capable of absorbing a single ranged hit before being eliminated. Upon reaching the farthest rank, a Warrior earns a promotion — the controlling player selects one of three upgrades: a Mage, an Archer, or an Assassin.
- **Mage:** A magic-wielding caster. Rather than moving diagonally, the Mage slides exclusively in the four orthogonal directions (forward, backward, left, right), advancing up to two squares per turn. It fires projectiles straight ahead in those same four directions, with a reach of two tiles.
- **Archmage:** An elite upgrade to the Mage. It shares the Mage's King-like movement but unleashes magic bolts with unlimited orthogonal range. Due to the power of this attack, a two-turn cooldown is imposed after each shot.
- **Archer:** A precision ranged unit. Unlike the Mage, the Archer moves exclusively along diagonals — up to two squares per turn. Its arrows fly diagonally as well, reaching targets up to two tiles away.
- **Assassin:** A stealthy flanking unit. It leaps diagonally over any obstacles in its path, covering up to two squares in a single bound.
- **Dragon:** An all-direction slider that replaces the Queen. It glides any number of squares horizontally, vertically, or diagonally and engages in melee combat by occupying the target's tile on a successful capture.

Prepared by:
Kasidis Chayachawalit | Student ID: 6831308121
Pitakwut Sanpanawat | Student ID: 6831353621

1.3 Combat & Capturing System

- **Melee Combat:** Non-ranged pieces attack by moving directly into an occupied square. A successful hit instantly eliminates the target, and the attacker takes its place on the board.
- **Ranged Combat:** Ranged units strike from a distance without leaving their square. Most targets are eliminated outright. The exception is a full-health Warrior — a ranged hit against one merely depletes its first life rather than destroying it.

2. How to Run the Program

This program is developed in Java and uses JavaFX for the Graphical User Interface (GUI). Therefore, the following environment setup is required:

2.1 Prerequisites

- **Java Development Kit (JDK):** Version 17 or higher.
- **JavaFX SDK:** Download the SDK and note the path to the lib folder.

2.2 Running via Command Line

Navigate to */build/libs* inside the project folder then execute the following command:

```
java --module-path "path/to/javafx/lib"  --add-modules javafx.controls,javafx.fxml,javafx.media -jar Kuroshiro.jar
```

Note: Before pressing Enter, you must replace the "path/to/javafx/lib" placeholder in the command with the exact, absolute path to the JavaFX lib folder on your specific computer. **Do not remove the quotation marks.**

2.3 Running via run.bat

A launch script (run.bat) is located in the /build/libs folder:

```
@echo off
java --module-path "path/to/javafx/lib" --add-modules javafx.controls,javafx.fxml,javafx.media -jar Kuroshiro.jar
pause
```

Note: Before running the .bat file, you must open it in a text editor (such as Notepad) and replace "path/to/javafx/lib" with the exact, absolute path to the JavaFX lib folder on your specific computer. **Do not remove the quotation marks.**

Prepared by:
Kasidis Chayachawalit | Student ID: 6831308121
Pitakwut Sanpanawat | Student ID: 6831353621

2.4 Running via IntelliJ IDEA

This project utilizes Gradle to automatically manage JavaFX dependencies and module paths. The recommended way to run the application is through IntelliJ IDEA:

1. Open the project folder in **IntelliJ IDEA**.
2. Allow a few moments for Gradle to sync and download necessary dependencies.
3. On the far right side of the IDE window, click to open the **Gradle** tool window.
4. Expand the project tree and navigate to: **Tasks > application**.
5. Double-click the **run** task to launch the game.

3. UI Controls & Interaction

3.1 Game Setup

The game board displays 64 squares in an 8x8 grid. White moves first, after which turns alternate between the two players. The white pieces start at the bottom, while the black pieces start at the top.

As illustrated in **Figure 3.1**, the setup is organized as follows:

- **The Frontline (Ranks 2 and 7):** Both players deploy a solid wall of **Warriors** across their respective second ranks This forms the initial defensive line.
- **The Backline (Ranks 1 and 8):** The specialized units are arranged on the back rank to protect the flanks and center:
    - ❖ **Corners (a and h files):** The **Assassins** are placed on the outermost squares.
    - ❖ **Outer Flanks (b and g files):** The **Archers** sit adjacent to the Assassins.
    - ❖ **Inner Flanks (c and f files):** The **Mages** are placed next to the Archers.
    - ❖ **The Center (d and e files):** The **Dragon** is positioned on the d-file while the **Archmage** is positioned on the e-file, replacing the usual Queen's and King's square respectively.

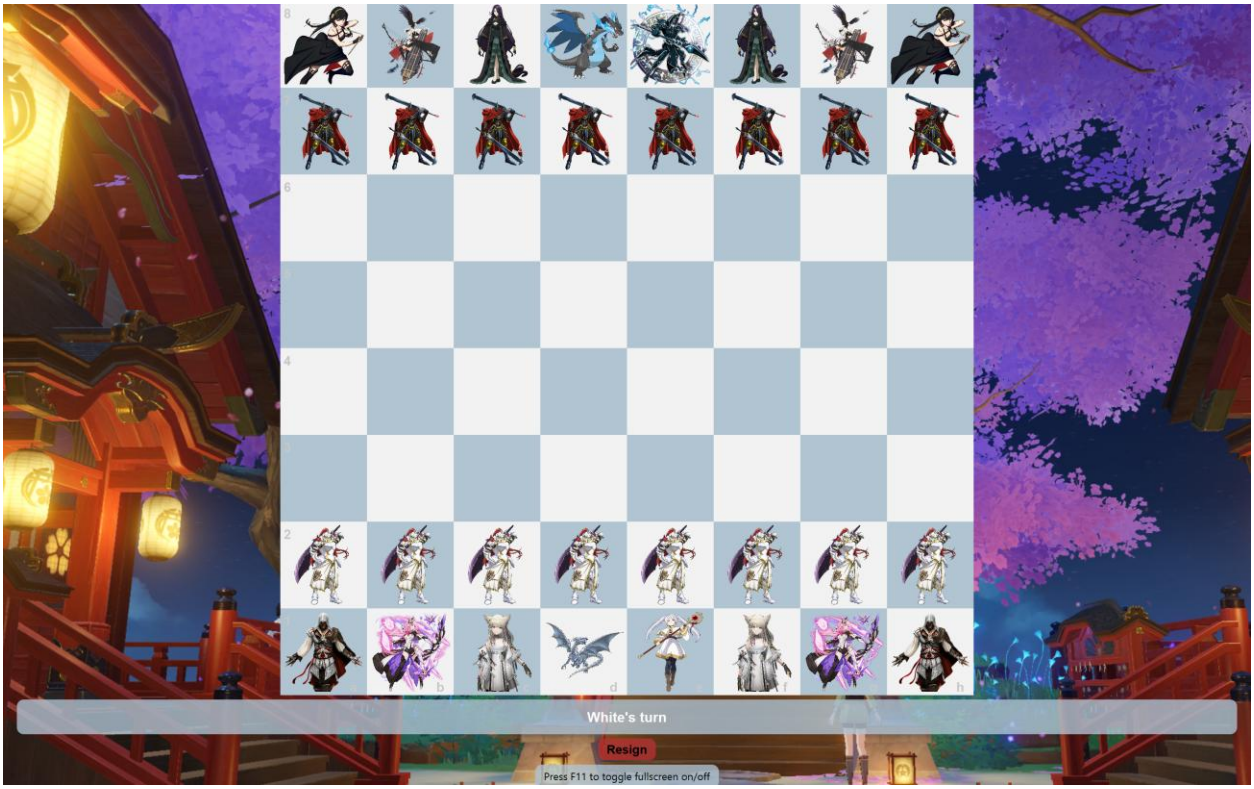

Figure 3.1 – Setup game board

3.2 Actions

Following is a list of actions you can do:

Action	Instruction
Select a piece	Click any piece. A yellow highlight marks the selection.
Standard Move	Click a green-highlighted square to move the piece there.
Combat Move	Orange squares appear for valid targets. Click one to initiate combat
Resign	Press the Resign button to concede the match.
New Game	Once the game concludes, a New Game button appears to restart.
Promote	Click the desired piece icon to complete the promotion (see **Figure 3.2**).

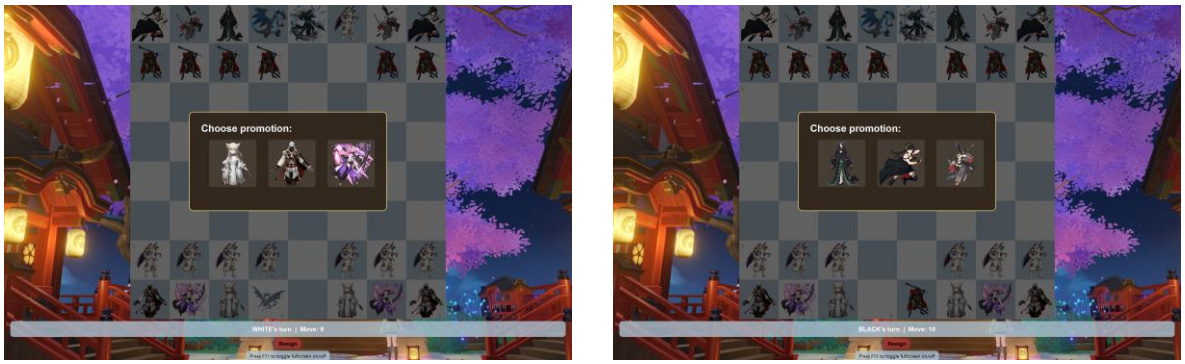

Figure 3.2 – Warrior Promotion UI

Prepared by:
Kasidis Chayachawalit | Student ID: 6831308121
Pitakwut Sanpanawat | Student ID: 6831353621

4. Javadoc Documentation

Detailed documentation for every class and method is available in the generated Javadoc. The full API reference can be accessed at the link: https://muanxng.github.io/Kuroshiro/

5. System Architecture (UML Diagrams)

The following diagrams illustrate the structure and relationships within the Kuroshiro engine.

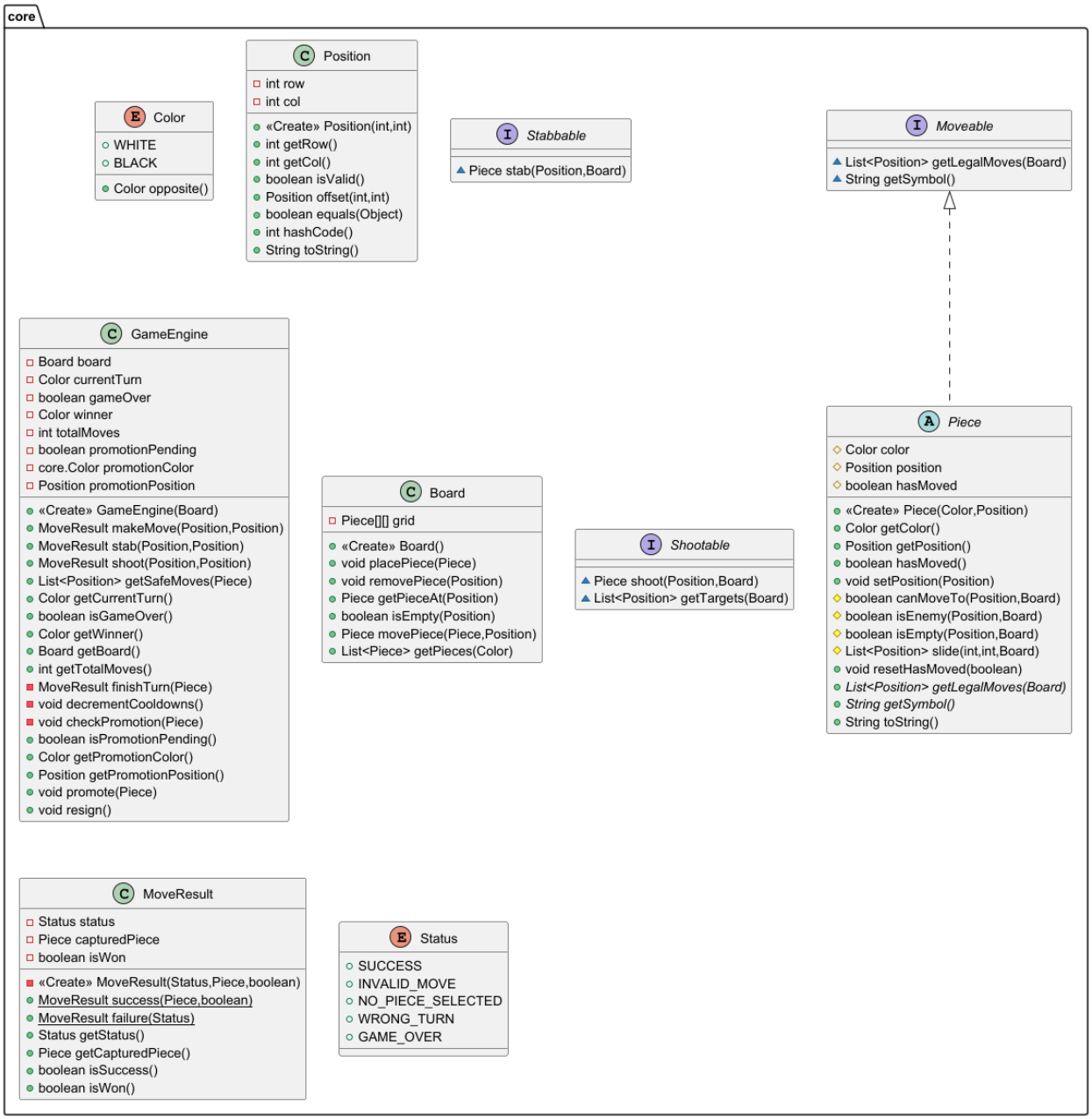

Figure 5.1 — Core package class diagram

Prepared by:
Kasidis Chayachawalit | Student ID: 6831308121
Pitakwut Sanpanawat | Student ID: 6831353621

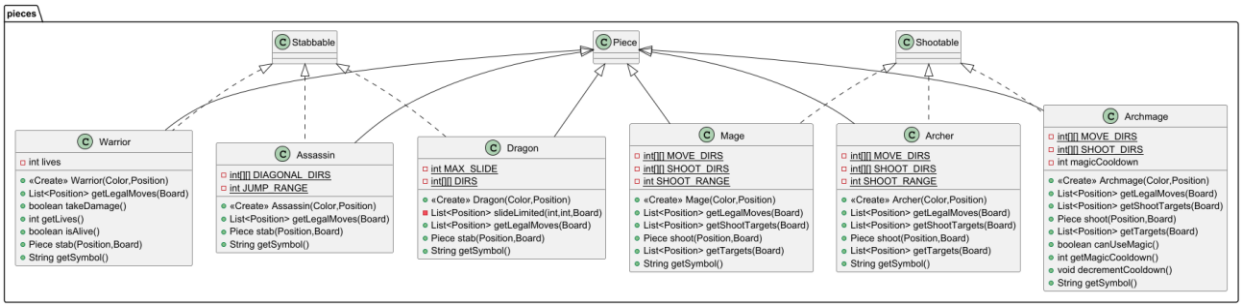

Figure 5.2 — Pieces package class diagram

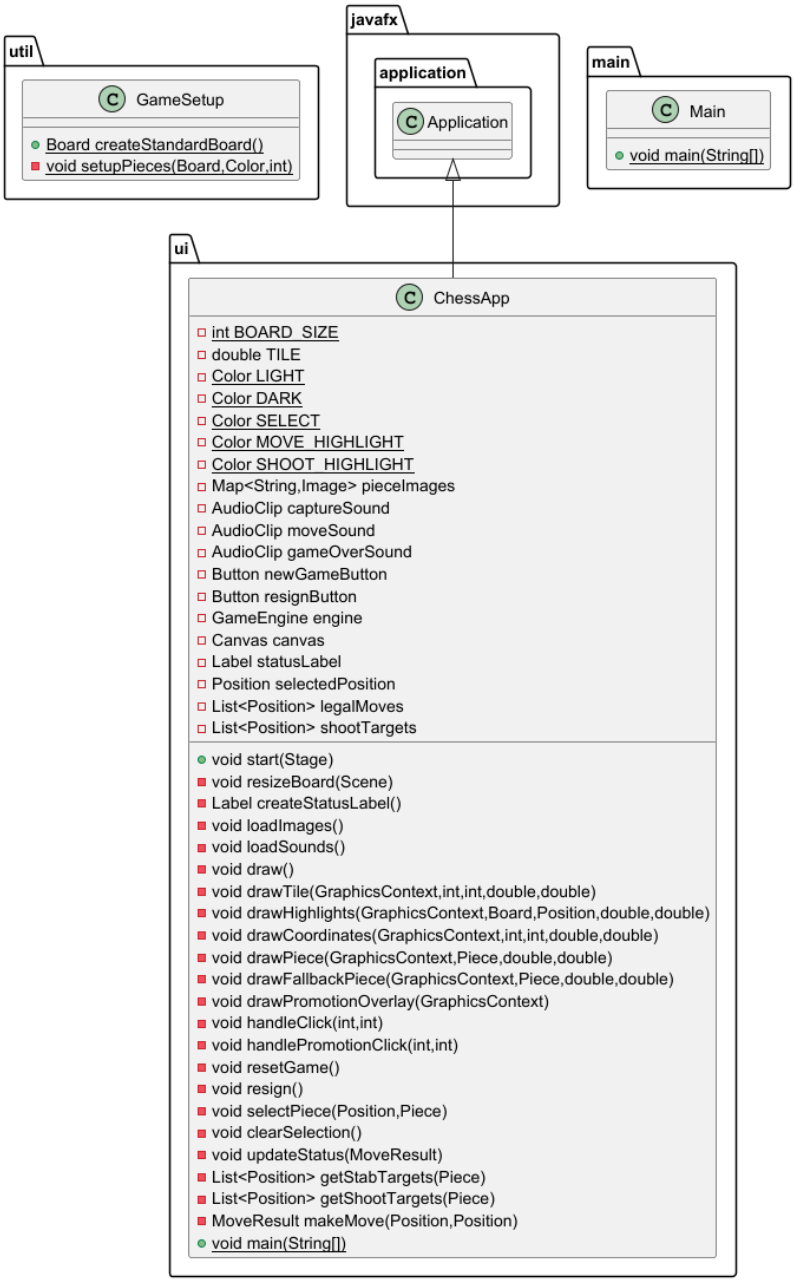

Figure 5.3 — UI, main, and util package diagram

Prepared by:
Kasidis Chayachawalit | Student ID: 6831308121
Pitakwut Sanpanawat | Student ID: 6831353621

Figure 5.4 — Test suite class diagram